

AI-Based Fake Image Detection System



Submitted by:

Huzaifa Amir / 23108113
M. Zulfiqar Ali / 23108130
Syed Hassnain Raza / 23108142

Submitted to:

Mr. Muhammad Azhar

**Department of Robotics and AI
SZABIST University Islamabad
Spring 2026**

1. Introduction

With the rapid advancement of artificial intelligence, especially generative models, the creation of highly realistic fake images (commonly known as deepfakes) has become increasingly easy. These manipulated images can be used for misinformation, identity fraud, and other malicious activities. Therefore, there is a growing need for intelligent systems that can automatically distinguish between real and fake images.

This project delivers a completed, end-to-end AI-based Fake Image Detection System that unifies advanced computer vision pipeline techniques with deep learning architectures. The system ingests, normalizes, processes, and analyzes visual patterns in uploaded portrait imagery, successfully classifying them as either real or fake via an interactive web interface and a programmatic execution layer.

2. Objectives

The completed objectives of this project are:

- To build and deploy a functional, automated end-to-end classification system to flag fake and deepfake portrait data.
- To implement a robust, reproducible preprocessing pipeline using localized classical computer vision spatial transformations.
- To train and compile a custom Convolutional Neural Network (CNN) specifically tailored to structural portrait features.
- To evaluate system performance using rigorous validation split criteria and track live operational metrics.
- To design, package, and launch an accessible, production-ready graphical web user interface alongside a command-line interface for multi-channel usage.

3. Methodology

The completed system implements a multi-stage production computer vision pipeline:

3.1 Data Collection & Sampling

- **Dataset Size & Splitting:** The total underlying dataset encompasses approximately 12,900 images partitioned into two explicit structural balance groups: ~7,000 Fake images and ~5,890 Real images.
- **Balanced Subsampling:** To accommodate computational constraints and eliminate class bias, a balanced subset of 4,000 total images (2,000 Fake / 2,000 Real) was extracted utilizing a fixed random seed to guarantee experimental reproducibility during training.

3.2 Automated Image Preprocessing Pipeline

Every input image passes through an identical sequential preprocessing chain to prevent data drift between training and inference cycles:

- **Contrast Enhancement (LAB Space):** Images are converted from BGR to LAB color space so that histogram equalization can be applied exclusively to the L-channel (luminance), boosting feature definition without distorting color channels.
- **Gamma Correction:** Power-law transforms are dynamically mapped through pre-computed lookup tables (cv2.LUT) to equalize varying exposure and illumination levels.
- **Dimensionality Standardization:** Images are uniformly downscaled to a fixed resolution of $128 * 128 * 3$.
- **Min-Max Normalization:** Pixel arrays are cast to floating-point values and scaled strictly between 0.0 and 1.0 ($X / 255.0$).

3.3 Computer Vision Diagnostic Modules

For visual verification and foundational edge analysis, specialized diagnostic blocks are integrated within the system environment:

- **Canny Edge Detection:** Exposes structural anomalies, artifacts, and blending irregularities found around high-frequency facial boundaries.
- **Image Negative Transformation:** Utilizes point-processing inversion to scrutinize illumination, compression blocks, and unnatural noise patterns across the canvas.

3.4 Deep Learning Model Architecture

A custom Sequential Convolutional Neural Network (CNN) was developed and trained from scratch:

- **Feature Extraction Layers:** Multiple Convolutional layers pass over the spatial inputs to map foundational shapes, micro-textures, and artifact boundaries.
- **Dimensionality Reduction:** Max-Pooling layers downsample feature space to compress parameter counts and retain dominant signals.
- **Classification Head:** Fully connected Dense layers receive the flattened feature matrices, feeding an optimized output node powered by Binary Crossentropy loss and the Adam optimization algorithm.

[Input: $128 \times 128 \times 3$] $\longrightarrow$ [Conv Layers + Pooling] $\longrightarrow$ [Dense Classification Head] $\longrightarrow$
[Binary Output: 0 (Real) / 1 (Fake)]

3.5 Web & CLI Deployment Architectures

The system has transitioned from a local script into a multi-channel accessible application:

- **Streamlit Web UI (app.py):** An interactive browser interface operating in wide-mode configuration. Users upload images via drag-and-drop, which are decoded natively in memory via OpenCV to minimize disk I/O latency.
- **Command Line Interface (predict.py):** A headless script that utilizes system arguments (sys.argv) to run high-speed predictions on local directories directly through the terminal console.

4. Features of the Final System

- **Real-Time Dual-Channel Inference:** Users can generate predictions instantly through either the graphical web app or the terminal CLI.
- **Zero-Disk In-Memory Processing:** Uploaded images are converted directly from raw bytes into OpenCV-compatible arrays to maximize runtime performance.
- **Interactive Visual Analytics:** Displays prediction certainties using dynamic metric widgets and real-time confidence tracking bars.
- **Deterministic Pipeline Execution:** Standardized caching (st.cache_resource) ensures the model weights are loaded once, maintaining fast response times across multiple predictions.

5. Feasibility and System Requirements

Technical Feasibility

The platform utilizes open-source, highly stable libraries, making it fully operational on standard, non-GPU hardware environments:

- **Runtime Environment:** Python 3.13
- **Deep Learning Framework:** TensorFlow / Keras 2.20+
- **Core Image Engine:** OpenCV-Python
- **UI Framework:** Streamlit

Operational Feasibility

The system features a highly intuitive user interface. Non-technical users simply drag and drop any standard portrait (JPG, JPEG, or PNG), and the system handles the preprocessing, validation, and final classification automatically.

6. Work Completed and Deliverables

All development phases have been successfully finalized. The completed project repository includes the following deliverables:

1. **train.ipynb**: The complete training and exploration notebook containing dataset downsampling, preprocessing validation, and CNN training.
2. **preprocessing.py & inference.py**: Core pipeline modules that house contrast-enhancement, gamma correction, and optimized inference engines.
3. **app.py & predict.py**: Ready-to-run Streamlit deployment application and native Command Line Interface execution scripts.
4. **setup.ps1 & run_app.ps1**: Automated PowerShell automation scripts to handle virtual environment provisioning and application initialization instantly.

7. Results and Performance Evaluation

The custom Convolutional Neural Network successfully achieved a baseline validation accuracy of **68%** within 5 training epochs over the balanced subset. The training routine effectively verified the code pipeline, proving that the preprocessing steps mirror perfectly between training and inference phases. This stable foundation ensures the application functions correctly without experiencing structural code errors or data mismatch issues.

8. Conclusion

This project successfully demonstrates a complete end-to-end application of computer vision and deep learning to address a critical security challenge. By embedding classical image preprocessing transformations seamlessly within a modern convolutional neural network, the system successfully classifies real and fake portraits in real time. The modular design of the codebase provides a production-ready baseline that can easily scale to accommodate larger datasets or more complex deep learning models in the future.

9. Contributions

Names	Contributions
Huzaifa Amir	Developed core application code, built the training notebook pipeline, and engineered the Streamlit UI and CLI modules.
M. Zulfiqar Ali	Managed technical documentation, verified architecture pipeline compliance, and audited structural codebase synchronization.
Syed Hasnain Raza	Designed final deployment review presentations, compiled evaluation metrics, and managed quality assurance checks.